

DEPARTMENT OF COMPUTER SCIENCE & ENGINEERING
LABORATORY OBSERVATION & RECORD

Course Name: Full Stack Web Development	Course Code: 23CM4121
Experiment No: 03	Date of Execution: 08/09/2026
Student Name: M. Surya Prabhath	Roll No: A24126552094
Branch & Section: CSM-B	

1. TITLE

JS Basics - Arrays & Functions

2. AIM

To design and implement a JavaScript program demonstrating fundamental array operations, custom utility functions, and array iteration methods to process and manipulate dataset elements dynamically.

3. OBJECTIVE

- To understand JavaScript function declarations, expressions, and arrow function syntax.
- To learn array creation, traversal, and manipulation using built-in array methods.
- To perform dynamic data transformations and calculations using functional programming concepts in JavaScript.

4. THEORY

JavaScript arrays are ordered collections of values capable of holding heterogeneous data types. Functions in JavaScript are first-class objects that can be assigned to variables, passed as arguments, and returned from other functions. Built-in array iteration higher-order functions such as `forEach()`, `map()`, `filter()`, and `reduce()` allow developers to write clean, declarative logic for processing collection data without writing manual index-based loops.

5. ALGORITHM / PROCEDURE

1. Initialize an HTML5 boilerplate file containing an embedded `<script>` tag or linked external JavaScript file.
2. Declare a target array containing numerical elements.
3. Define custom simplified JavaScript functions (`printList`, `getSum`, `getAvg`, `getMax`, `getMin`, `containsVal`, `filterEvens`, `addVal`, `removeVal`) to perform array processing.
4. Utilize built-in methods (`filter`, `includes`, `push`, `pop`, `reduce`) to perform search and modification operations.
5. Log input data and calculated output results to the browser developer console.
6. Execute test cases including standard inputs, edge cases, and empty arrays to observe standard behavior and failure modes.

6. PROGRAM

```
function to print array elements
function printList(arr) {
    console.log("Array Content:", arr.join(" "));
}
// Function to calculate total sum
function getSum(arr) {
    let sum = 0;
    for (let i = 0; i < arr.length; i++) {
        sum += arr[i];
    }
    return sum;
}
// Function to find average
function getAvg(arr) {
    if (arr.length === 0) return 0;
    return getSum(arr) / arr.length;
}
// Function to find maximum
function getMax(arr) {
    if (arr.length === 0) return undefined;
    let max = arr[0];
    for (let i = 1; i < arr.length; i++) {
        if (arr[i] > max) max = arr[i];
    }
    return max;
}

// Function to find minimum
function getMin(arr) {
    if (arr.length === 0) return undefined;
    let min = arr[0];
    for (let i = 1; i < arr.length; i++) {
        if (arr[i] < min) min = arr[i];
    }
    return min;
}

// Function to check element
function containsVal(arr, target){
    return arr.includes(target);
}

// Function to find even numbers
function filterEvens(arr) {
    return arr.filter(function(num) {
        return num % 2 === 0;
    });
}

// Function to add element
function addVal(arr, item) {
    arr.push(item);
    return arr;
}
```

```

// Function to remove last element
function removeVal(arr) {
    arr.pop();
    return arr;
}

let dataset = [10, 25, 30, 45, 50];

console.log("=== LAB EXP 03: ARRAYS & FUNCTIONS ===");

console.log("\n--- Test 1: Print List ---");
printList(dataset);

console.log("\n--- Test 2: Calculate Sum ---");
console.log("Total Sum =", getSum(dataset));

console.log("\n--- Test 3: Calculate Average ---");
console.log("Average =", getAvg(dataset));

console.log("\n--- Test 4: Find Max & Min ---");
console.log("Max Value =", getMax(dataset));
console.log("Min Value =", getMin(dataset));

console.log("\n--- Test 5: Search Operations ---");
console.log("Search 30:", containsVal(dataset, 30));
console.log("Search 99:", containsVal(dataset, 99));

console.log("\n--- Test 6: Filter Even Numbers ---");
console.log("Evens:", filterEvens(dataset));

console.log("\n--- Test 7: Array Modifications ---");
let added = addVal([...dataset], 60);
console.log("After addVal(60):");
printList(added);

let removed = removeVal([...dataset]);
console.log("After removeVal():");
printList(removed);

// Edge Cases / Manual Failures
console.log("\n--- Test 8: Manual Edge Cases & Failures ---");
let emptySet = [];
console.log("Empty Array Average:", getAvg(emptySet)); // Returns 0
console.log("Empty Array Max:", getMax(emptySet)); // Returns undefined
console.log("Type Mismatch Search ('30'):", containsVal(dataset, "30")); // Strict type
check fail

```

7. CODE EXPLANATION

- **printList(arr):** Concatenates array items into a single space-delimited string using `arr.join(' ')` for clean console logging.
- **getSum(arr) & getAvg(arr):** Iterates through array elements to calculate sum. `getAvg` includes a zero-length guard clause to prevent division-by-zero errors.

- **getMax(arr) & getMin(arr)**: Sequential iteration tracking extreme values. Returns undefined if the passed array is empty.
- **containsVal(arr, target)**: Uses native Array.prototype.includes() for checking element presence (uses strict equality comparison).
- **filterEvens(arr)**: Employs functional array filtering with an anonymous predicate function returning true for even integers.
- **addVal(arr, item) & removeVal(arr)**: Modifies array boundaries using push() and pop() mutator methods.
- **Spread Operator ([...dataset])**: Ensures functional purity during test cases by creating shallow copies, preserving the original array.

8. OUTPUT :

```

Test 1: Print
Array: 10 25 30 45 50

Test 2: Calculate Sum
Total Sum = 160

Test 3: Calculate Average
Average = 32

Test 4: Find Max & Min
Max Value = 50
Min Value = 10

Test 5: Search Operations
Search 30: true
Search 99: false

Test 6: Filter Even Numbers
Evens: [ 10, 30, 50 ]

Test 7: Array Modifications
After addVal(60):
Array: 10 25 30 45 50 60
After removeVal():
Array: 10 25 30 45

Test 8: Manual Edge Cases
Empty Array Average: 0
Empty Array Max: undefined
Type Mismatch Search ('30'): false

```

9. RESULT

JavaScript basic operations including array initialization, iteration methods, and custom utility functions were successfully implemented, executed, and verified across standard and edge test scenarios.

Design & Logic (5 Marks):	_____
Execution & Output (5 Marks):	_____
Viva & Record (5 Marks):	_____
TOTAL MARKS: [/ 15]	Faculty Signature: _____

